PALM Code Structure & Features

palmgroup

Institute of Meteorology and Climatology, Leibniz Universität Hannover

General remarks

- This lecture gives a brief overview about the code structure of PALM.

PALM Code Structure & Features

Overview

- PALM is written in FORTRAN2008.
- With some very minor exceptions, the code is using the FORTRAN standard, so it should compile without error on any FORTRAN 2003/2008 compiler (90/95 may give problems).
- Machine dependent code segments, e.g. calls of routines from external libraries (e.g. netCDF or FFTW), which may not be available on some machines, are activated using preprocessor directives.
- The serial and parallel (MPI) PALM version is also activated by preprocessor directives.
- The installer automatically sets the approriate preprocessor directives. For manual settings of directives see https://palm.muk.uni-hannover.de/trac/wiki/doc/app/cpp_options.

PALM Code Structure & Features

Overview

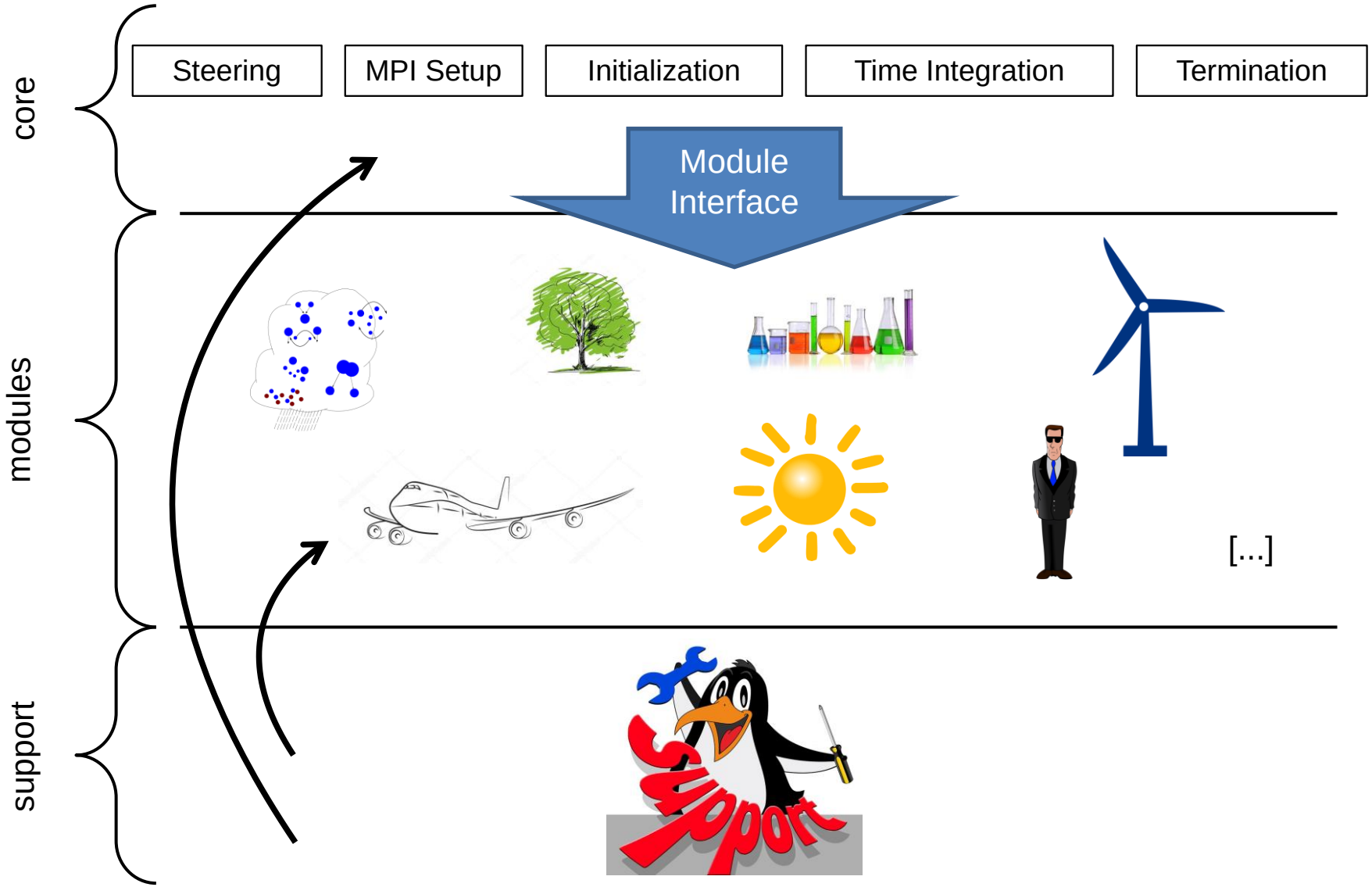
- The code is split into multiple files, each file containing:
  - a single `MODULE` (ending with `_mod.f90`), including several associated SUBROUTINEs, or
  - a single `SUBROUTINE`, e.g. file `parin.f90` contains `SUBROUTINE parin`.
- PALM includes a special user module (`user_module.f90`) designed to add additional code written by the user.

Why should you use the user module instead of directly modiying the source code?

- The user module very rarely changes in future PALM releases and can be easily re-used by newer versions of PALM without requiring extensive changes.

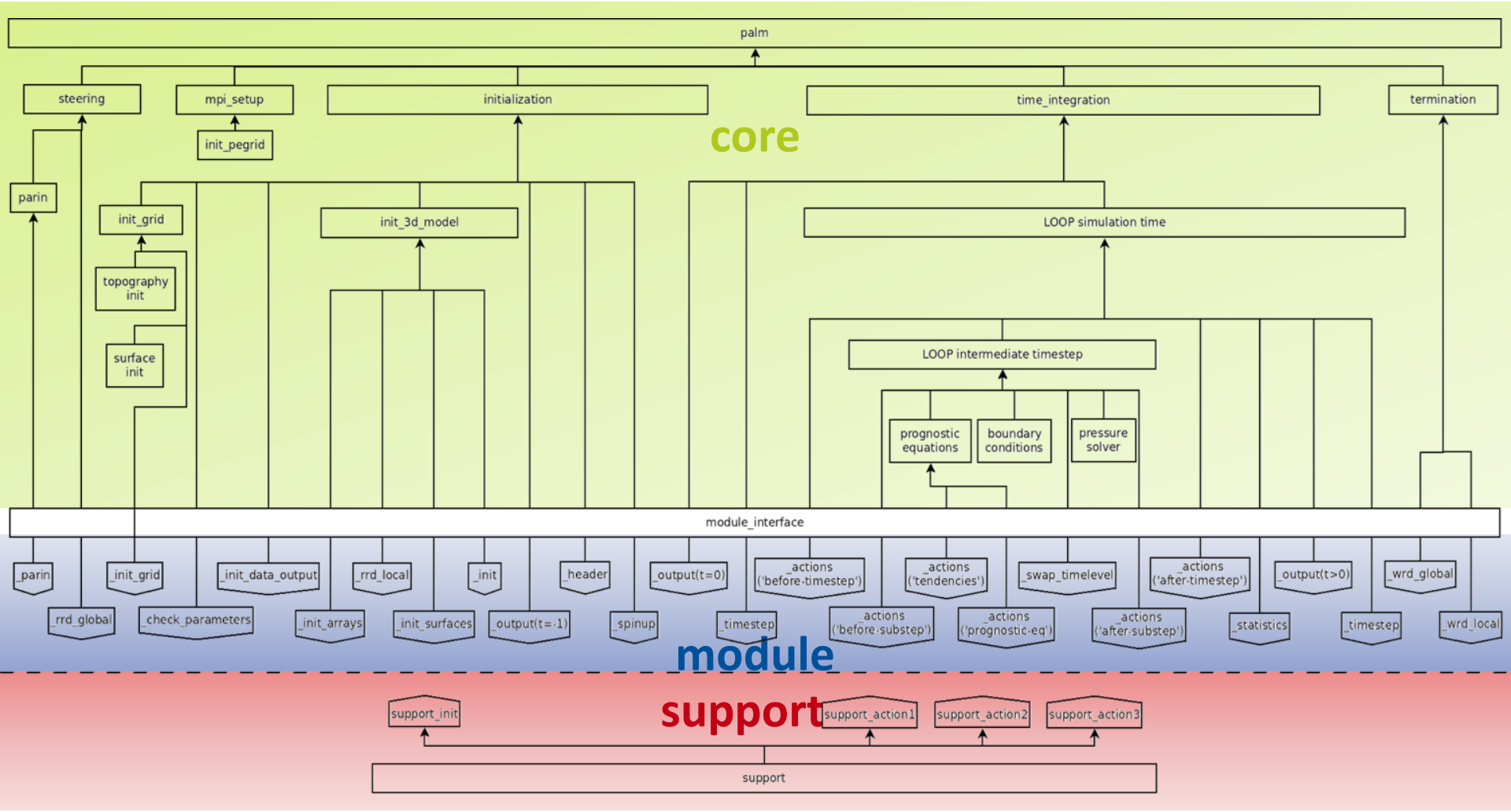
PALM Code Structure & Features

General structure



PALM Code Structure & Features

Code structure



PALM Code Structure & Features

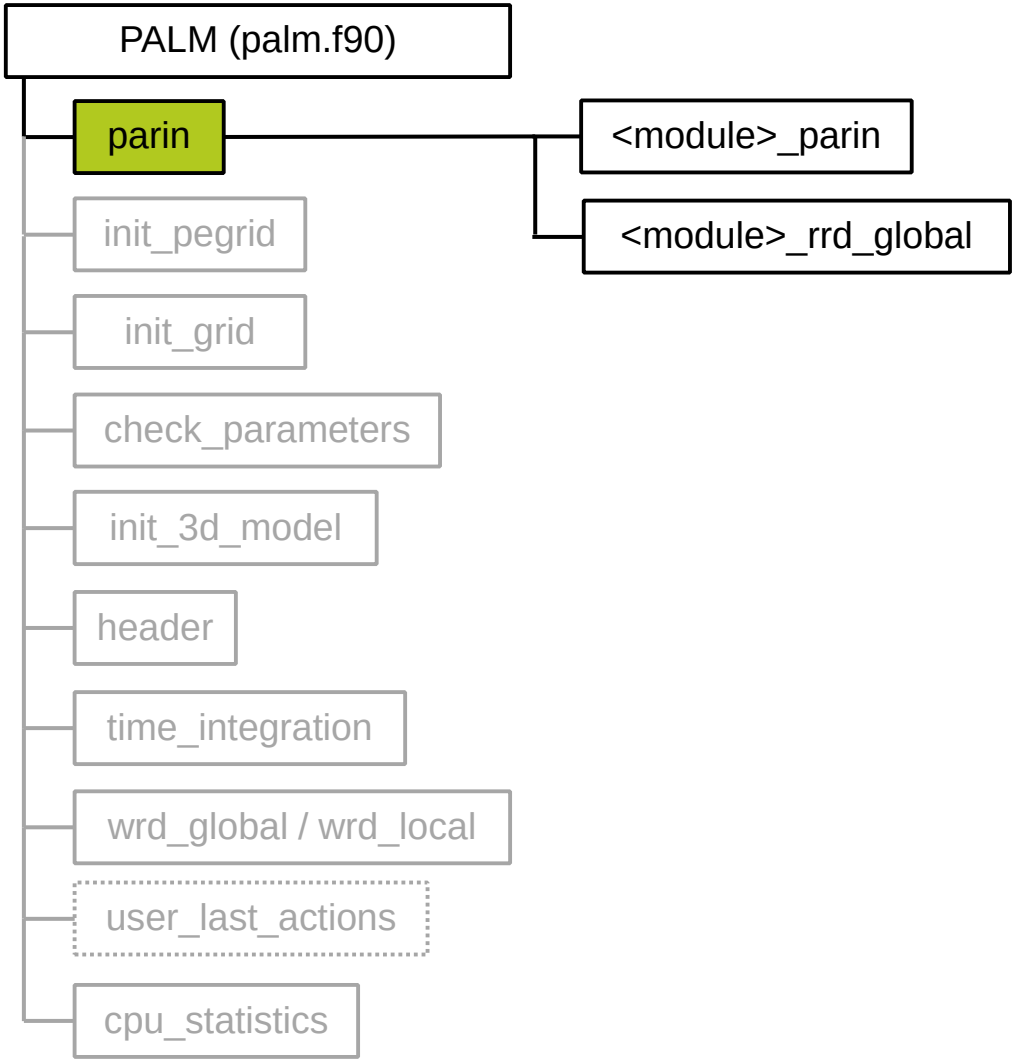
Detailed structure

- PALM (palm.f90)
  - parin
  - init_pegrid
  - init_grid
  - check_parameters
  - init_3d_model
  - header
  - time_integration
  - wrd_global / wrd_local
  - user_last_actions
  - cpu_statistics

- Read steering parameters
- Create processor topology (for parallel PALM mode)
- Create grid-depending constants
- Check setting & dependencies of steering parameters
- Allocate arrays and initialize 3D model
- Output important information about current run
- Integrate model equations (statistical analysis, data output)
- Binary output for restarts
- Execute user-defined last actions (if user module is used)
- Output of performance measurements (cpu times)

PALM Code Structure & Features

Detailed structure



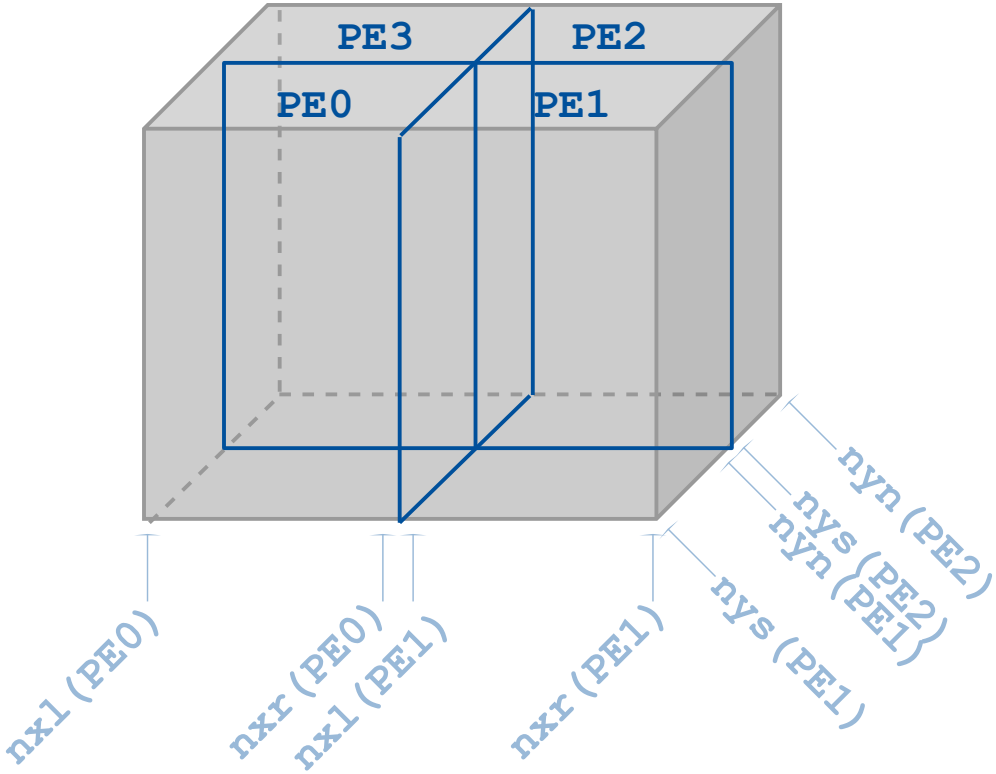
- Read input parameters from namelist file
- Read control parameters from restart file in case of restart run

PALM Code Structure & Features

Detailed structure

- PALM (palm.f90)
  - parin
  - **init_pegrid**
  - init_grid
  - check_parameters
  - init_3d_model
  - header
  - time_integration
  - wrd_global / wrd_local
  - user_last_actions
  - cpu_statistics

- Determination of virtual processor topology (if not prescribed by user) & computation of grid point number and array bounds of local subdomains

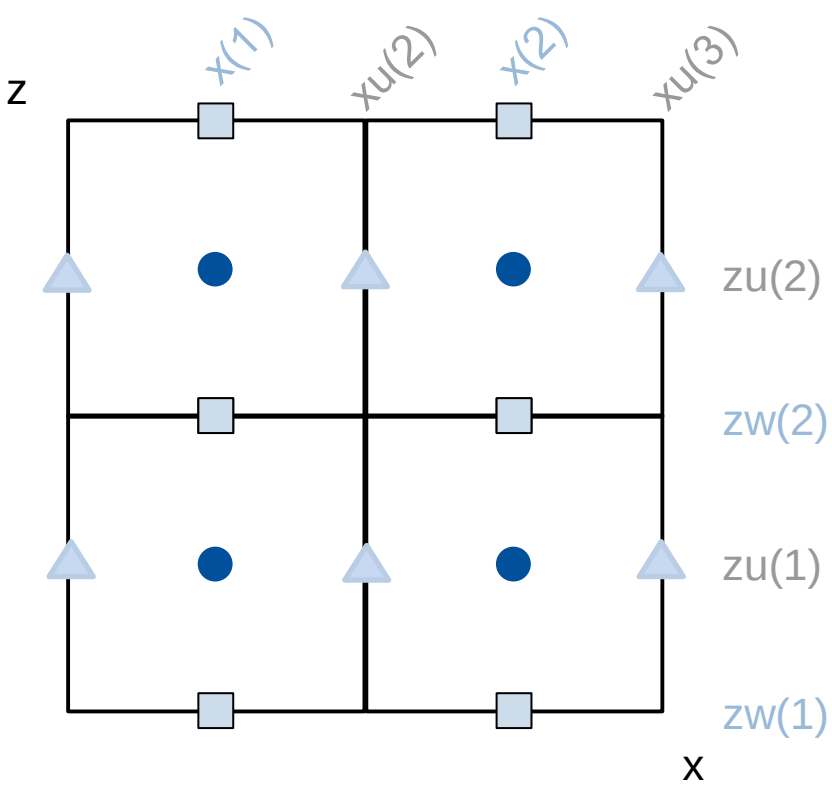


PALM Code Structure & Features

Detailed structure

- PALM (palm.f90)
  - parin
  - init_pegrid
  - **init_grid**
  - check_parameters
  - init_3d_model
  - header
  - time_integration
  - wrd_global / wrd_local
  - user_last_actions
  - cpu_statistics

- Pre-calculation of metric grid coordinates on staggered PALM grid, e.g.:



PALM Code Structure & Features

Detailed structure

- PALM (palm.f90)
  - parin
  - init_pegrid
  - init_grid
  - check_parameters
    - <module>_check_parameters
    - <module>_check_data_output_ts
    - <module>_check_data_output_pr
    - <module>_check_data_output
  - init_3d_model
  - header
  - time_integration
  - wrd_global / wrd_local
  - user_last_actions
  - cpu_statistics

- Check simulation setup for any inconsistencies
- Check output setup

In case of any problems, error messages appear in the job protocol, labeled with a specific PALM error code number “**PA...**.“.

Details about error messages can be found at:
https://https://docs.palm-model.org/25.04/Reference/LES_Model/Logging/

PALM Code Structure & Features

Detailed structure

- PALM (palm.f90)
  - parin
  - init_pegrid
  - init_grid
  - check_parameters
  - **init_3d_model**
    - <module>_init_arrays — Initialize module-specific arrays
    - init_1d_model — Pre-calculate profiles via 1D model
    - init_surfaces — Initialize surfaces
    - <module>_rrd_local — Read arrays from restart data files
    - ls_forcing_surf — Load large-scale forcing
    - disturb_field — Impose random perturbations
    - pres — Call pressure solver
    - <module>_init — Initialization actions of the modules
    - ws_init — Initialize Wicker-Skamarock scheme
  - header
  - time_integration
  - wrd_global / wrd_local
  - user_last_actions
  - cpu_statistics

PALM Code Structure & Features

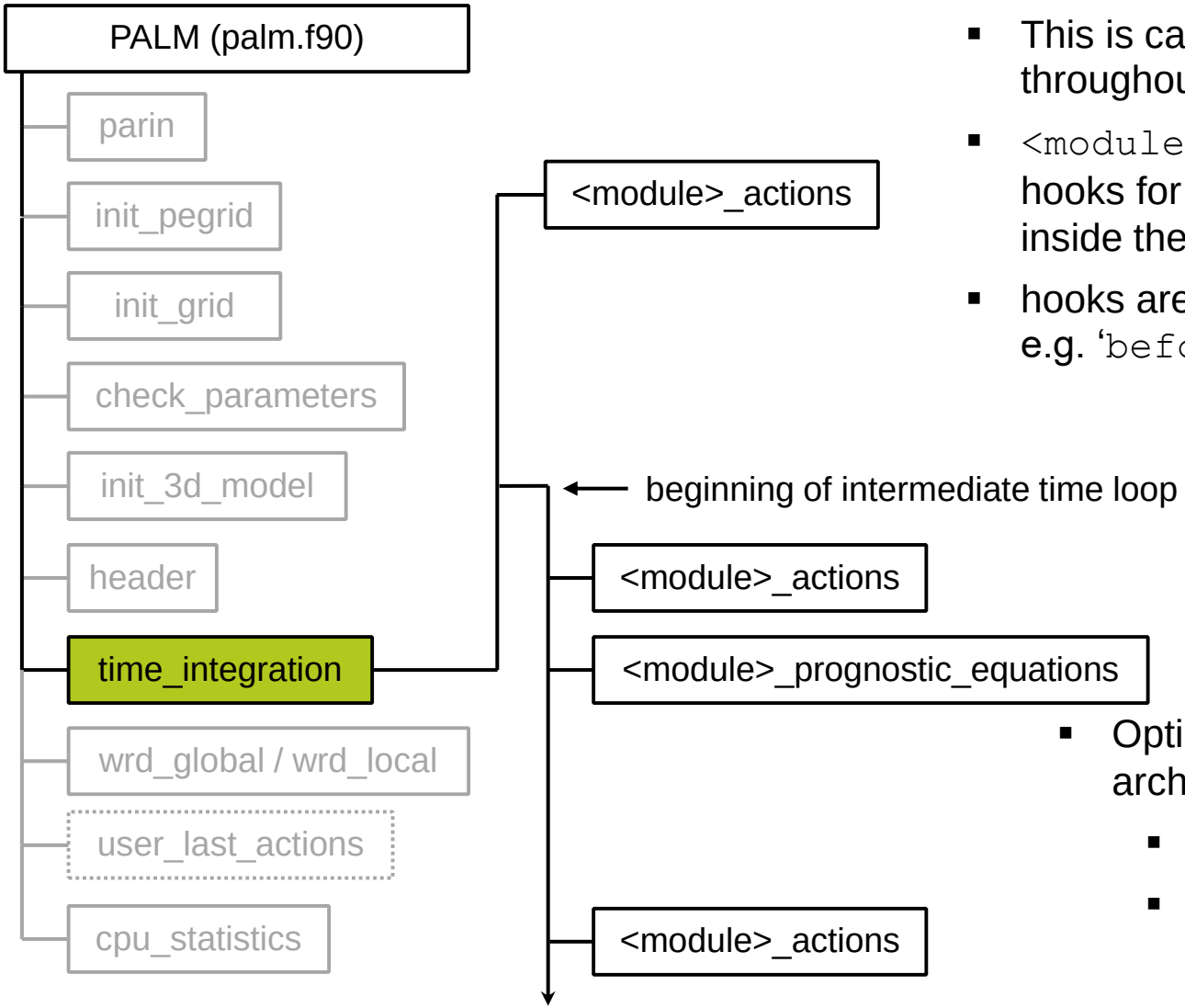
Detailed structure

- PALM (palm.f90)
  - parin
  - init_pegrid
  - init_grid
  - check_parameters
  - init_3d_model
  - **header**
  - time_integration
  - wrd_global / wrd_local
  - user_last_actions
  - cpu_statistics

- Write information about steering parameters to file, useful for job monitoring (see lecture “PALM steering”)

PALM Code Structure & Features

Detailed structure



- This is called at several positions throughout `time_integration`
- `<module>_action` has several hooks for different positions inside the time loop
- hooks are addressed via strings, e.g. '`before_timestep`'

- Optimized for different computer architectures
  - cache-based machines
  - vector-based machines

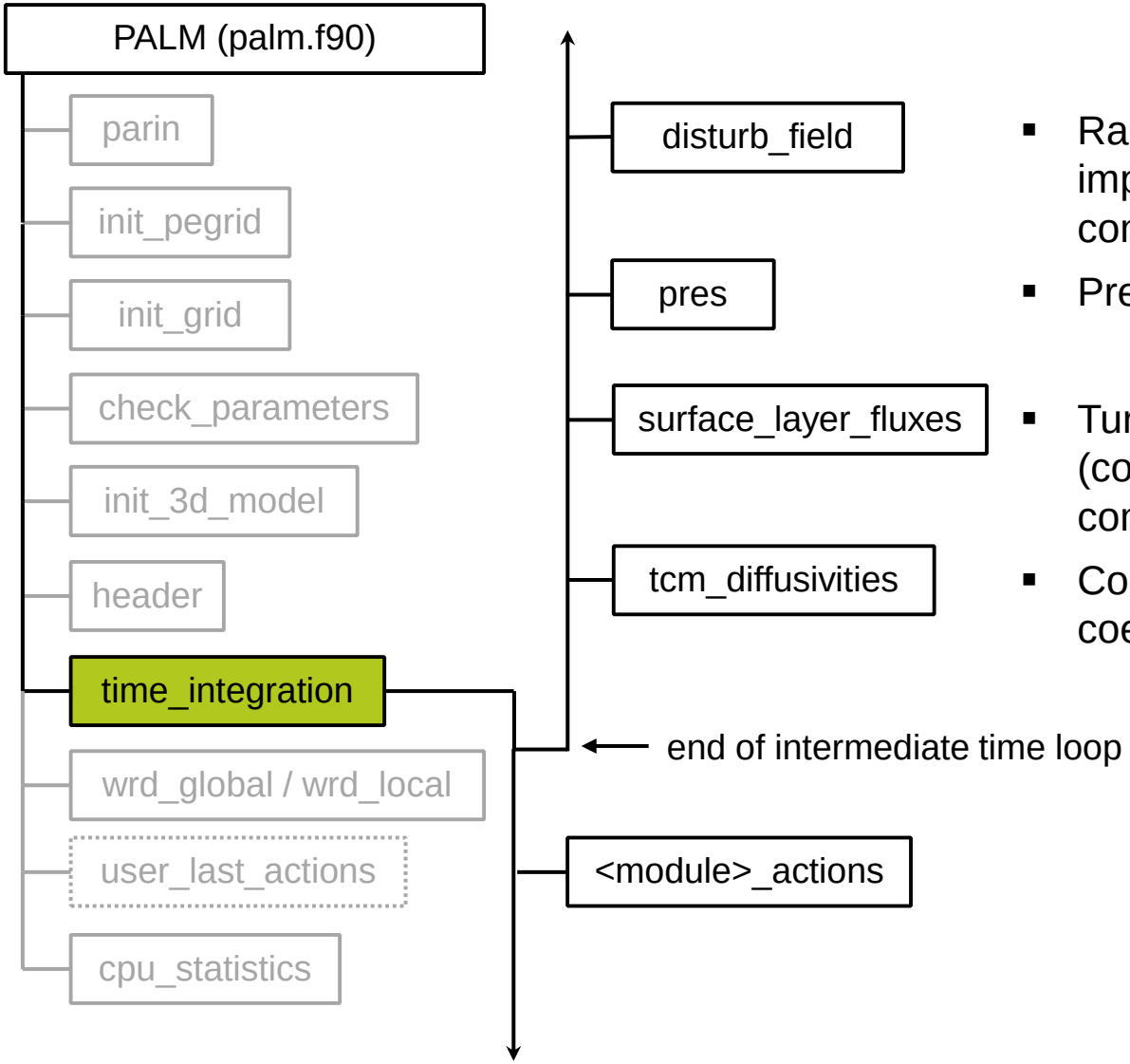
PALM Code Structure & Features

Detailed structure

- PALM (palm.f90)
  - parin
  - init_pegrid
  - init_grid
  - check_parameters
  - init_3d_model
  - header
  - **time_integration**
    - exchange_horiz
      - Exchange of ghost-point data between neighbouring PEs
      - Also called at some other locations
    - boundary_conds
      - Setting of boundary conditions
    - <module>_swap_timelevel
      - Swapping of prognostic quantities data
    - inflow/outflow_turbulence
      - Calculating turbulent inflow and/or outflow conditions
  - wrd_global / wrd_local
  - user_last_actions
  - cpu_statistics

PALM Code Structure & Features

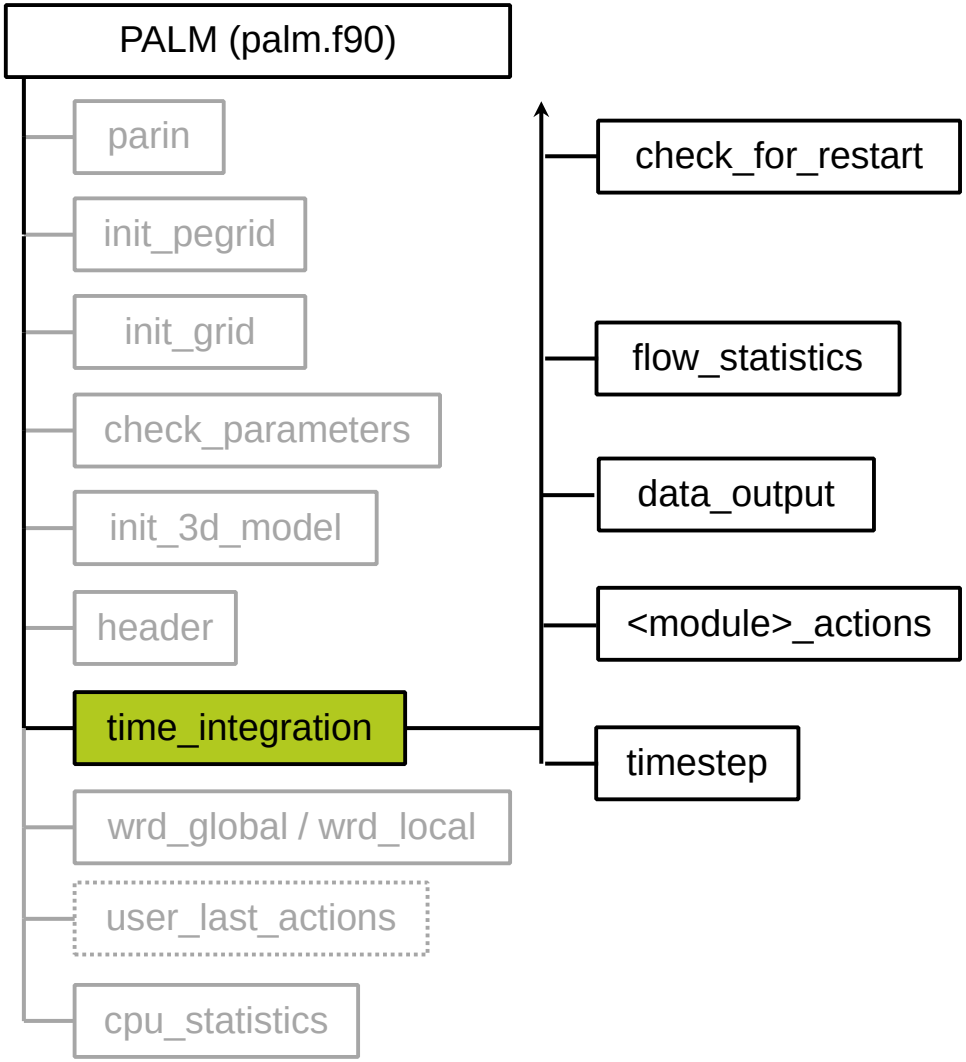
Detailed structure



- Random perturbations are imposed to horizontal velocity components
- Pressure solver
- Turbulent fluxes in the surface (constant-flux) layer are computed
- Computing of diffusion coefficients

PALM Code Structure & Features

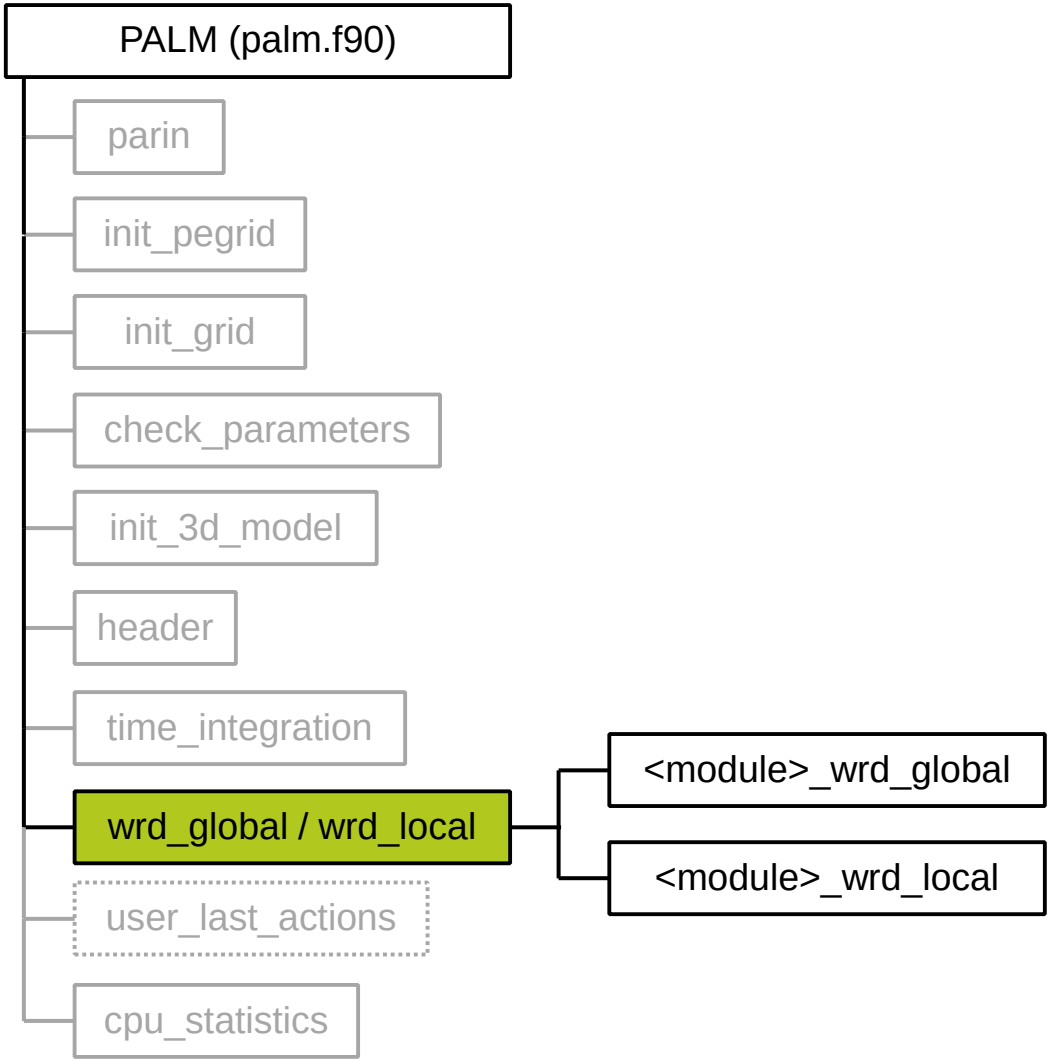
Detailed structure



- Check, if run needs to be terminated (due to insufficient remaining CPU time) and prepared for restart
- Flow statistics are calculated
- Output of requested variables
- Calculating next timestep width

PALM Code Structure & Features

Detailed structure



- Write data required for restarts
  - Steering parameters
  - 3d arrays of prognostic variables

PALM Code Structure & Features

Detailed structure

- PALM (palm.f90)
  - parin
  - init_pegrid
  - init_grid
  - check_parameters
  - init_3d_model
  - header
  - time_integration
  - wrd_global / wrd_local
  - **user_last_actions**
  - cpu_statistics

- Any other last actions before end of simulation

PALM Code Structure & Features

Detailed structure

- PALM (palm.f90)
  - parin
  - init_pegrid
  - init_grid
  - check_parameters
  - init_3d_model
  - header
  - time_integration
  - wrd_global / wrd_local
  - user_last_actions
  - **cpu_statistics**

- Calculate and output information about required CPU time, in total and for certain parts of PALM, e.g. “all progn. equations” or “pres” (see lecture “PALM steering”)

Content – Part 2

- Global variables
- Preprocessor directives
- Automatic documentation using Doxygen
- Important variables and their declaration

Global variables

- Global variables and parameters are defined in **`modules.f90`**
- Contains only those variables that are used in multiple parts of PALM
- Defined in different **`MODULE`**s, e.g., **`pt`** is defined in **`arrays_3d`**:

```
MODULE arrays_3d
[...]
    REAL(wp), DIMENSION(:,:,:), ALLOCATABLE, TARGET :: pt
[...]
END MODULE
```

Global variables

- **USE** statements to access module variables in other routines:

```
SUBROUTINE buoyancy( var, wind_component )

    USE arrays_3d,                                  &
        ONLY:  pt, pt_slope_ref, ref_state, tend

    USE control_parameters,                         &
        ONLY:  atmos_ocean_sign, cos_alpha_surface
[...]

END SUBROUTINE buoyancy
```

Preprocessor directives

- Preprocessor directives are special lines in the code which allow to compile alternative parts of the code depending on so-called "**define string** switches"
- Code example:

```
#if defined ( __parallel )
    CALL MPI_ALLREDUCE( a, b, nzt-nzb, MPI_REAL, MPI_SUM,
                        comm2d, ierr )
#else
    b = a
#endif
```

- If the compiler is called as
  `ifx -fpp -D__parallel ...`
  then the **#if** branch is compiled
- If the compiler is called **without** option `-D__parallel`
  the **#else** branch is compiled

Preprocessor directives

- The preprocessor directives require to activate their processing with a specific compiler option, which is e.g. **`-fpp`** for the Intel compiler.
- Preprocessor directives and options have to be given in the **`%cpp_options`** line of the palmrun configuration file **`(.palm.config.<ci>`**, see lecture "PALM steering")
- Preprocessor options may differ for different compilers

Additional use of preprocessor directives

- Replacing strings in the code, e.g.,

```
%cpp_options -fpp -DMPI_REAL=MPI_DOUBLE_PRECISION
```

replaces `MPI_REAL` with `MPI_DOUBLE_PRECISION` **before** compiling

List of define-string switches used in PALM

PALM mode	**`__parallel`**	Parallel PALM version
	`__single_precision`	Use 32-bit arithmetic (still in test phase)
System specific	**`__nec`**	NEC-SX systems
	`__cuda_fft`	FFT-library for GPU
Software specific	**`__intel_compiler`**	Compilers
	`__netcdf, __netcdf4, __netcdf4_parallel`	NetCDF I/O with different NetCDF versions
	`__netcdf_compression`	NetCDF compression libraries
	`__fftw`	Fast FFT
	`__rrtmg`	External radiation model library (see lecture)

- Switches that are set under **`%cpp_options`** in **`.palm.config.<ci>`** file are automatically used by **`palmbuild`** for compiling.